

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Exp. No: 6

Date: 14-08-2026

Generation of Intermediate Code using Lex and Yacc Tools

Aim:

To build an intermediate code generator with Lex and Yacc that produces three address code for assignment statements and boolean expressions, using syntax directed translation rules.

Tools Used:

- Flex
- Bison
- GCC

Code:

tac.l

```
%{
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "y.tab.h"

char stmt_text[256];          /* source text of the statement just read */
static char srcline[256];    /* characters of the statement being read */

#define YY_USER_ACTION  strncat(srcline, yytext, sizeof(srcline) - strlen(srcline) - 1);

/* Move srcline into stmt_text (without the terminator) and start afresh */
static void end_statement(void)
{
    char *s = srcline;
    int n;
    while (isspace((unsigned char)*s)) s++;
    strcpy(stmt_text, s);
    n = strlen(stmt_text);
    while (n > 0 && (isspace((unsigned char)stmt_text[n-1]) || stmt_text[n-1] == ';' ))
        stmt_text[--n] = '\0';
    srcline[0] = '\0';
}

static int pending(void)
{
    char *s;
    for (s = srcline; *s; s++)
        if (!isspace((unsigned char)*s)) return 1;
    return 0;
}
}%

%option yylineno

DIGIT  [0-9]
LETTER [a-zA-Z_]

%%

"or"|"||" { return OR; }
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

"and"|"&&"           { return AND; }
"not"|"!"            { return NOT; }

{LETTER}({LETTER}|{DIGIT})*
{DIGIT}+(\.{DIGIT}+)? { yylval.str = strdup(yytext); return ID; }
                      { yylval.str = strdup(yytext); return NUM; }

":="|"="            { return ASSIGN; }
"<="|">="|"=="|"!="|"<"|>" { yylval.str = strdup(yytext); return RELOP; }
[+ \- * / % ()]      { return yytext[0]; }

[ \n; ]              { end_statement(); return EOL; }
[ \t \r ]+           ;
.                    { return yytext[0]; }

<<EOF>>              { if (pending()) { end_statement(); return EOL; }
                      return 0; }

%%

int yywrap(void)
{
    return 1;
}

```

tac.y

```

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

int yylex(void);
void yyerror(const char *s);

extern int yylineno;
extern char stmt_text[];

#define BASE 100          /* number given to the first instruction */
#define MAXCODE 100

static char code[MAXCODE][80]; /* three address code of the current statement */
static int ncode = 0;          /* instructions generated so far */
static int ntemp = 0;          /* temporaries used so far */
static int hasBool = 0;        /* statement contains a boolean expression */

static char *newtemp(void)
{
    char buf[16];
    sprintf(buf, "t%d", ++ntemp);
    return strdup(buf);
}

static void emit(const char *fmt, ...)
{
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(code[ncode++], sizeof(code[0]), fmt, ap);
    va_end(ap);
}

static void reset(void)
{
    ncode = ntemp = hasBool = 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

/* Print the code of one statement: numbered from 100 only if it has booleans */
static void flush_statement(void)
{
    int k;
    printf("%s\n", stmt_text);
    for (k = 0; k < (int)strlen(stmt_text); k++) putchar('-');
    putchar('\n');
    for (k = 0; k < ncode; k++) {
        if (hasBool) printf("%d: %s\n", BASE + k, code[k]);
        else         printf("%s\n", code[k]);
    }
    putchar('\n');
    reset();
}

/* result = x op y for arithmetic operators */
static char *arith(char *x, char op, char *y)
{
    char *t = newtemp();
    emit("%s = %s %c %s", t, x, op, y);
    return t;
}

/* result = x relop y : four instructions giving the value 0 or 1 */
static char *relational(char *x, char *op, char *y)
{
    char *t = newtemp();
    int a = BASE + ncode; /* number of the 'if' instruction */
    hasBool = 1;
    emit("if %s %s %s goto %d", x, op, y, a + 3);
    emit("%s := 0", t);
    emit("goto %d", a + 4);
    emit("%s := 1", t);
    return t;
}

/* result = x and y / x or y */
static char *logical(char *x, const char *op, char *y)
{
    char *t = newtemp();
    hasBool = 1;
    emit("%s := %s %s %s", t, x, op, y);
    return t;
}
%}

%union { char *str; }

%token <str> ID NUM RELOP
%token ASSIGN OR AND NOT EOL

%type <str> expr

%left OR
%left AND
%right NOT
%nonassoc RELOP
%left '+' '-'
%left '*' '/' '%'
%right UMINUS

%%

program : program stmt
        |
        ;

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

stmt      : ID ASSIGN expr EOL      { emit("%s = %s", $1, $3); flush_statement(); }
          | EOL
          | error EOL               { reset(); yyerrorok; }
          ;

expr       : expr OR expr           { $$ = logical($1, "or", $3); }
          | expr AND expr          { $$ = logical($1, "and", $3); }
          | NOT expr               { $$ = newtemp(); hasBool = 1;
                                   emit("%s := not %s", $$, $2); }
          | expr RELOP expr        { $$ = relational($1, $2, $3); }
          | expr '+' expr          { $$ = arith($1, '+', $3); }
          | expr '-' expr          { $$ = arith($1, '-', $3); }
          | expr '*' expr          { $$ = arith($1, '*', $3); }
          | expr '/' expr          { $$ = arith($1, '/', $3); }
          | expr '%' expr          { $$ = arith($1, '%', $3); }
          | '-' expr %prec UMINUS { $$ = newtemp(); emit("%s = -%s", $$, $2); }
          | '(' expr ')'           { $$ = $2; }
          | ID                     { $$ = $1; }
          | NUM                    { $$ = $1; }
          ;

%%

void yyerror(const char *s)
{
    printf("Syntax error at line %d\n\n", yylineno);
}

int main(void)
{
    yyparse();
    return 0;
}

```

Input (input.txt)

```

a := b * -c + b* -c
x:= a<b or c<d and e<f

```

Commands Used

```

lex tac.l
yacc -d tac.y
cc lex.yy.c y.tab.c -lfl -o tac
./tac < input.txt

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment 6 % flex tac.l
● kushaal@Kushaals-MacBook-Air Assignment 6 % yacc -d tac.y
● kushaal@Kushaals-MacBook-Air Assignment 6 % gcc lex.yy.c y.tab.c -o tac
● kushaal@Kushaals-MacBook-Air Assignment 6 % ./tac < input.txt
a := b * -c + b* -c
-----
t1 = -c
t2 = b * t1
t3 = -c
t4 = b * t3
t5 = t2 + t4
a = t5

x:= a<b or c<d and e<f
-----
100: if a < b goto 103
101: t1 := 0
102: goto 104
103: t1 := 1
104: if c < d goto 107
105: t2 := 0
106: goto 108
107: t2 := 1
108: if e < f goto 111
109: t3 := 0
110: goto 112
111: t3 := 1
112: t4 := t2 and t3
113: t5 := t1 or t4
114: x = t5

```

Three address code generated for the given input

The temporaries t1, t2, ... restart for each statement. A statement containing a boolean expression is numbered from 100, and a relational expression $a < b$ becomes four instructions: if $a < b$ goto (n+3), $t := 0$, goto (n+4), $t := 1$. Operator priority from lowest to highest is or, and, not, relational, arithmetic, so in $a < b$ or $c < d$ and $e < f$ the and is evaluated before the or.

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Additional Test Cases:

```

• kushaal@Kushaals-MacBook-Air Assignment 6 % cat input2.txt
y := (a + b) * (c - d) / 2
z := not a < b and c >= d
p := a + b * c < d - 1
q := a +* b
• kushaal@Kushaals-MacBook-Air Assignment 6 % ./tac < input2.txt
y := (a + b) * (c - d) / 2
-----
t1 = a + b
t2 = c - d
t3 = t1 * t2
t4 = t3 / 2
y = t4

z := not a < b and c >= d
-----
100: if a < b goto 103
101: t1 := 0
102: goto 104
103: t1 := 1
104: t2 := not t1
105: if c >= d goto 108
106: t3 := 0
107: goto 109
108: t3 := 1
109: t4 := t2 and t3
110: z = t4

p := a + b * c < d - 1
-----
100: t1 = b * c
101: t2 = a + t1
102: t3 = d - 1
103: if t2 < t3 goto 106
104: t4 := 0
105: goto 107
106: t4 := 1
107: p = t4

Syntax error at line 4

```

More test cases (the last statement contains a syntax error)

Learning Outcomes:

- Learned how syntax directed translation attaches semantic actions to grammar rules, so three address code is produced while the input is parsed.
- Understood how the attributes (\$\$, \$1, \$3) carry the name of the temporary holding each sub-expression's value, and how new temporaries are created.
- Learned to translate relational expressions into conditional jumps with instruction numbers, and to combine them using and / or / not.
- Used precedence declarations to give unary minus, arithmetic, relational and logical operators the right priority, and the yacc error token to recover from a bad line.